

Simple Attention Architecture

Mritunjoy Kar Pial
pialadittoo@gmail.com, mritunjoy.kar.pial@ulkasemi.com

May 2026

Architecture Overview

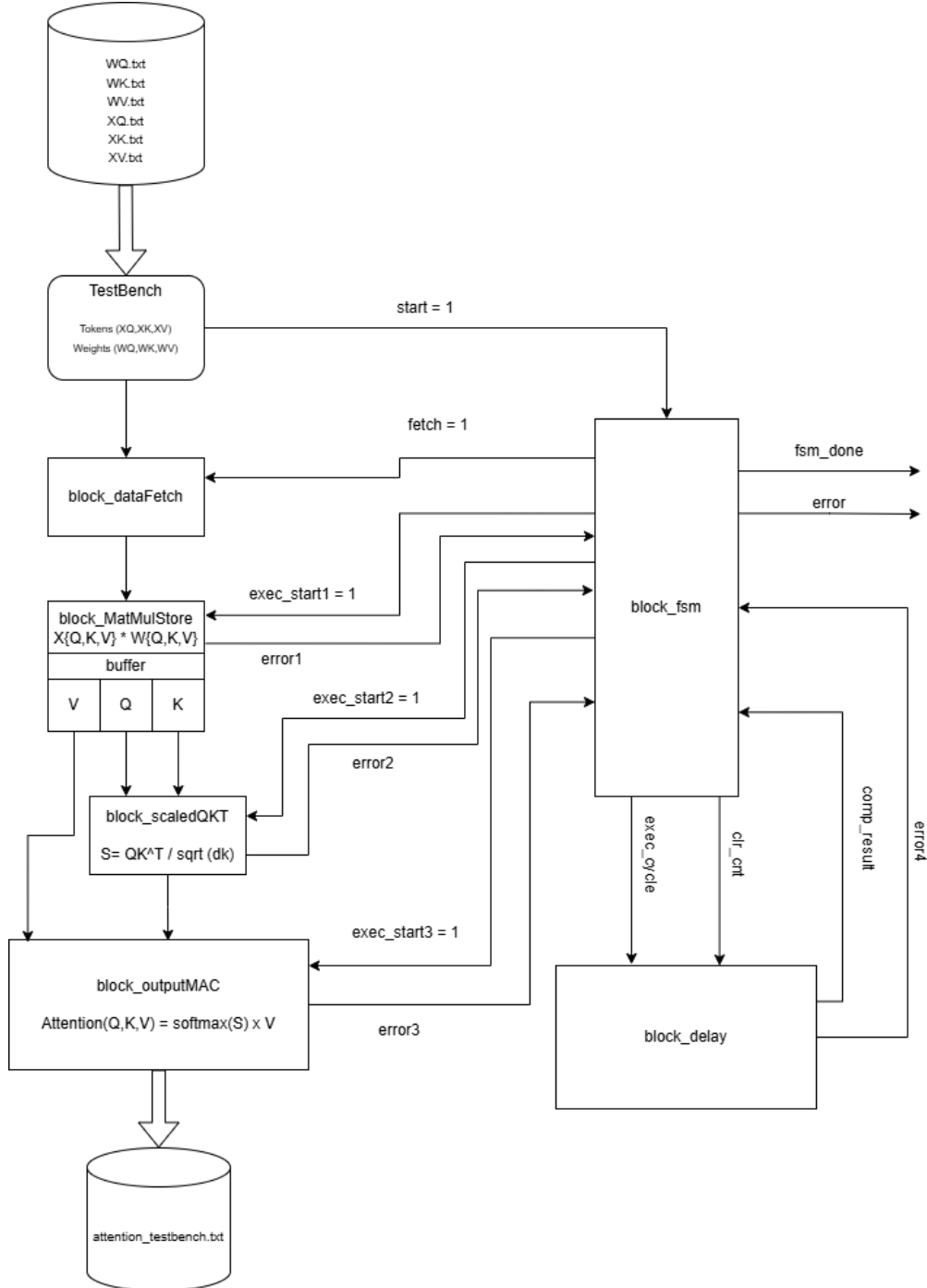


Figure 1: Architecture diagram

Scaling Unit

Newton-Raphson Method to find $QK^T/\sqrt{d_k}$
Newton-Raphson iteration:

$$S_{n+1} = S_n - \frac{f(S_n)}{f'(S_n)}, \quad \text{where } f(S_n) = 0$$

Here, $S_n = \frac{1}{\sqrt{d_k}}$
or, $d_k - \frac{1}{S_n^2} = 0$

Hence, $f(S_n) = d_k - \frac{1}{S_n^2}$ and $f'(S_n) = \frac{2}{S_n^3}$

Putting values to the equation:

$$S_{n+1} = S_n - \frac{d_k - \frac{1}{S_n^2}}{\frac{2}{S_n^3}}$$

Simplified:

$$S_{n+1} = S_n (1.5 - 0.5 \cdot d_k \cdot S_n^2)$$

SystemVerilog Implementation of scale_factor = $1/\sqrt{d_k}$:

```
dk = DK <<< 'FRAC_POINT;
for (int i = 1 ; i <= 'SCALE_NR_STEPS ; i ++ ) begin
    // Small value (less than 0.5) taken for the first iteration, since scale_factor approximately will be less than 1.
    if (i == 1) scale_factor = (1 <<< ('FRAC_POINT-3));
    else scale_factor = scale_NewtonRaphson( dk, scale_factor);
end

function automatic logic signed [DATA_WIDTH-1:0] scale_NewtonRaphson (
    input logic signed [DATA_WIDTH-1:0] dk,
    input logic signed [DATA_WIDTH-1:0] scale
);
    logic signed [DATA_WIDTH-1:0] scale_sq;
    scale_sq = ( scale * scale ) >>> 'FRAC_POINT;
    <----- 1.5 -----> <----- Q2m.2n to Qm.n --- +1 for 0.5 -->
    return ( scale * ( (3 <<< ('FRAC_POINT-1)) - ( ( dk * scale_sq ) >>> ('FRAC_POINT + 1) ) ) ) >>> 'FRAC_POINT;
endfunction
```

Limitations of Scaling Unit

Initial guess of first iteration is $1/2^3 = 0.125$. However, iterations can behave chaotically in case of large d_k , when $0.125 \gg 1/\sqrt{d_k}$. It may diverge from the root, approaching toward infinity.

Besides, if initial guess is too small than $1/\sqrt{d_k}$, there will be gradual growth which may take a lot of iteration steps, increasing execution time.

Softmax Normalization Unit

Softmax approximation of matrix $\text{mat_S}_{m \times n} = \frac{QK^T}{\sqrt{d_k}}$

Softmax equation:

$$\text{Softmax}(X_{ij}) = \frac{e^{x_{ij}}}{\sum_{j=1}^n e^{x_{ij}}}$$

Here, X_{ij} are the elements of the matrix $\text{mat_S}_{m \times n}[\text{ROW}][\text{COL}]$. Since the elements of the matrix may be large, which may lead to overflow after implementing e^x , largest element of each row has to be subtracted from each element of that row.

$$\text{norm_X}_{ij} = x_{ij} - \text{row_max}_i$$

SystemVerilog Pseudo-code implementation of softmax(mat_S) :

```
for (i = 0; i < ROW; i++) begin
    // Finding row max for numerical stability, since e^x value potentially will be too large to cause overflow.
    row_max = mat_S[i][0];
    for (j = 1; j < COL; j++) begin
        if ( mat_S[i][j] > row_max ) row_max = mat_S[i][j];
    end

    for (j = 0; j < COL; j++) begin
        // Normalized matrix element
        norm_x = mat_S[i][j] - row_max;

        Softmax(Xij) =  $\frac{e^{\text{norm\_x}_{ij}}}{\sum_{j=1}^n e^{\text{norm\_x}_{ij}}}$ ;
    end
end
```

Taylor series for exponential function e^x

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

However, since e^x takes more terms/steps to produce precise value for larger x by increasing the hardware complexity, in order to make it hardware friendly, lets break e^x such way that,

$$x = n \cdot \ln 2 + r$$

n = quotient (integer part)

ln2 = divisor part

r = reminder (small leftover value)

Hence,

$$e^x = e^{n \cdot \ln 2 + r} = 2^n \cdot e^r$$

Calculation of 2^n is easy to implement inside hardware via bit shifting. On the other hand, "r" becomes small, making Taylor series more stable with lesser steps.

Step.1: Compute $n = \frac{x}{\ln(2)}$

Step.2: Compute $r = x - n \cdot \ln(2)$

Step.3: Find 2nd part, sum = e^r .

```
// PART_2 : Taylor series for e^r = 1+ r + r^2/2! + ...
//-----

// Initiate first term of e^r:
sum  = 1 <<< 'FRAC_POINT;
power = 1 <<< 'FRAC_POINT;

// Calculate the rest of the terms of the series
for ( i = 1; i < SERIES_LENGTH; i++) begin
    power = (power*r) >>> 'FRAC_POINT;
    term  = (power <<< 'FRAC_POINT) / factorial(i);
    term  = term >>> 'FRAC_POINT;
    sum  = sum + term;
end
```

Step.4: For the first part 2^n , multiply e^r by 2^n .

```
// PART_1 : Scale result by 2^n (just bit shift)
//-----
if (n >= 0) return sum <<< n ; // Multiplying by 2^n
else      return sum >>> (-n); // Dividing by 2^(-n) = Multiplying by 2^n
```

Limitations of Softmax Normalization Unit

The time complexity of the Taylor series of e^x is always $O(N)$, where N is the number of terms. However, real performance depends on how each term is computed. In my design, each step performs:

- 1 multiplication: $\text{power} = \text{power} \times r$
- 1 division: $\text{term} = \frac{\text{power}}{\text{factorial}(i)}$
- 1 addition: $\text{sum} = \text{sum} + \text{term}$

It introduces heavy hardware cost per term due to $\text{factorial}(i)$ as the denominator.

Better approach to use recursive function to reduce hardware load of multipliers and $\text{factorial}(i)$ divisions in each steps,

```
term( 0 ) = 1  
term( i ) = ( term( i - 1 ) * r ) / i    ;   i >= 1  
sum = sum + term( i )
```

Design Constraints and Solutions

Timing Requirements

1. The matrix MAC operation is computed instantly in combinational modules in simulation. However, in real hardware, combinational logic is not instantaneous and requires finite propagation delay for each operation.
2. The Newton-Raphson and Taylor series approximations of e^x also require multiple computational steps. While simulation may produce results within a single clock cycle, in practical implementations this can lead to timing violations if the total propagation delay exceeds the allocated execution cycle of the *block_fsm* states.

These issues can be solved by choosing proper execution cycles in the *./rtl/define.sv* file,

```
/*-----  
      FSM State Exec Cycle  
-----*/  
'define EXEC_IDLE           1  
'define EXEC_DATA_FETCH    1  
'define EXEC_MATRIX_MULT   1  
'define EXEC_SCALED_ATTEN_SCORE 1  
'define EXEC_OUTPUT_MAC    1
```

Limited Data-Width

1. There is a potential risk of overflow/underflow during intermediate computations such as multiplication, accumulation, and exponential approximation. In this design, up-casting is used where it is needed to handle bit growth. Nevertheless, user has to choose the correct DATA_WIDTH in the *./rtl/define.sv* file based on worst possible intermediate growth in matrix elements, particularly in QK^T accumulation.

```
'define DATA_WIDTH 32
```